

### ■ 4.1.7 Rule-Based Programming

One of the most powerful, and unique, features of *Mathematica* is the ability to set up calculations by specifying collections of transformation rules. The basic idea is to say that whenever *Mathematica* sees an expression that matches a particular pattern, it should transform it in a particular way.

This approach allows you to write out a sequence of rules that mimic quite closely the tables of mathematical relations that you find in books.

```
diff[x_^n_, x] := n x^(n-1)

diff[Log[x], x] := 1/x
```

Two simple rules for differentiation.

```
diff[y_, x_] :=
  Block[ { n } ,
    If[ TrueQ [ Length[y] == 2 && y[[0]] == Power && y[[1]] == x ] ,
      n = y[[2]] ; n x^(n-1) ,
      If[ TrueQ[ y == x ] , 1 ,
        If[ TrueQ[ Length[y] == 1 && y[[0]] == Log && y[[1]] == x ] ,
          1/x ]
      ]
    ]
  ]
```

An ugly program that implements the simple differentiation rules using conditionals.

If you are trying to implement a sequence of mathematical rules or relations, rule-based programming is overwhelmingly the best method to use. The example above shows how complicated it is to imitate even a simple set of rules using explicit conditionals.

It is also worth realizing that the internal *Mathematica* code for transformation rules is highly optimized, and will usually work much faster than, for example, a collection of explicit conditionals.

There are many problems that are most naturally formulated in terms of transformation rules. When you analyze a problem, you will often find it better to start from scratch and build up a set of *Mathematica* rules than to take an existing procedural program, and adapt it to *Mathematica*.

A great advantage of rule-based programming is that it makes it easy for your to build up “programs” in stages. You start off with a small number of rules, and then progressively add more rules, to deal with wider and wider ranges of cases.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: [info@wolfram.com](mailto:info@wolfram.com); <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

```

bubble[{}] := {}

bubble[{single_}] := {single}

bubble[{first_, rest_}] := bubble1[{}, {first, rest}]

bubble1[head_, {last_}] := bubble2[bubble[head], last]

bubble1[{head___}, {first_, second_, rest___}] :=
    bubble1[{head, first}, {second, rest}] /; OrderedQ[{first, second}]

bubble1[{head___}, {first_, second_, rest___}] :=
    bubble1[{head, second}, {first, rest}]

bubble2[{head___}, last_] := {head, last}

```

A bubble sort program, written entirely with transformation rules.

As a matter of principle, it is not difficult to prove that *any Mathematica* program can in fact be implemented using transformation rules alone. In practice, not all programs are best formulated like this.

One guideline is that rule-based programming is appropriate when you can set up a large number of rules, but need to use only a few of them in any particular instance. If all the rules have to be used all the time, you may do better to write a more traditional procedural or functional program.